

Лабораторная работа № 9

Тема: Лабораторная работа №9. Интеграция REST API с клиентским приложением

Цель работы: Подключить браузерный frontend к разработанному Product REST API и реализовать из интерфейса получение, создание, изменение и удаление товаров.

Практическое задание

Продолжите проект **Internet Shop API**. Backend из предыдущих лабораторных работ сохраняется. Необходимо добавить каталог public и создать простой пользовательский интерфейс для управления товарами.

Итоговая структура:

```
express-shop/
├── controllers/
├── errors/
├── middleware/
├── routes/
├── services/
├── validators/
├── public/
│   ├── index.html
│   ├── css/
│   │   └── style.css
│   └── js/
│       ├── api.js
│       ├── ui.js
│       └── app.js
├── app.js
└── package.json
```

1. Подключите статические файлы

В серверном app.js добавьте:

```
app.use(express.json());
app.use(express.static("public"));
```

```
app.use("/api/products", productRoutes);
```

```
app.use(notFound);
```

```
app.use(errorHandler);
```

express.static() является встроенным middleware Express и может отдавать HTML, CSS и клиентские JavaScript-файлы из каталога public.

После запуска сервера откройте:

http://localhost:3000

2. Создайте public/index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Internet Shop</title>
  <link
    rel="stylesheet"
    href="/css/style.css"
  >
</head>

<body>
  <main>
    <h1>Internet Shop</h1>

    <section>
      <h2>Добавить товар</h2>

      <form id="product-form">
        <input
          id="name"
          placeholder="Название"
          required
        >

        <input
          id="price"
          type="number"
          placeholder="Цена"
          required
        >

        <input
          id="category"
          placeholder="Категория"
          required
        >
```

```

    <label>
      <input
        id="inStock"
        type="checkbox"
      >
      В наличии
    </label>

    <button type="submit">
      Добавить
    </button>
  </form>
</section>

<p id="message"></p>

<section>
  <h2>Товары</h2>
  <div id="products"></div>
</section>
</main>

<script
  type="module"
  src="/js/app.js"
></script>
</body>
</html>

```

3. Создайте `public/js/api.js`

В этом файле разместите только функции взаимодействия с REST API.

```

async function request(
  url,
  options = {}
) {
  const response =
    await fetch(url, options);

  let data = null;

  if (response.status !== 204) {
    data = await response.json();
  }
}

```

```

    }

    if (!response.ok) {
      throw new Error(
        data?.message ||
        `HTTP error ${response.status}`
      );
    }

    return data;
  }

```

Важно: `fetch()` не считает HTTP-ответ 404 или 400 сетевой ошибкой и возвращает объект `Response`. Поэтому HTTP-результат необходимо проверять через `response.ok` или `response.status`. `Response.json()` также является асинхронной операцией.

4. Реализуйте GET

В `api.js`:

```

export function getProducts() {
  return request("/api/products");
}

```

5. Реализуйте POST

```

export function createProduct(
  product
) {
  return request(
    "/api/products",
    {
      method: "POST",
      headers: {
        "Content-Type":
          "application/json"
      },
      body:
        JSON.stringify(product)
    }
  );
}

```

Для JSON-запроса тело преобразуется через `JSON.stringify()`, а заголовок `Content-Type` указывает `application/json`.

6. Реализуйте PATCH

```

export function updateProduct(

```

```

    id,
    changes
  ) {
    return request(
      `/api/products/${id}`,
      {
        method: "PATCH",
        headers: {
          "Content-Type":
            "application/json"
        },
        body:
          JSON.stringify(changes)
      }
    );
  }
}

```

7. Реализуйте DELETE

```

export function deleteProduct(id) {
  return request(
    `/api/products/${id}`,
    {
      method: "DELETE"
    }
  );
}

```

В результате `api.js` не должен содержать работу с DOM.

8. Создайте `public/js/ui.js`

```

const productsContainer =
  document.querySelector("#products");

const messageElement =
  document.querySelector("#message");

export function showMessage(text) {
  messageElement.textContent = text;
}

export function showError(text) {
  messageElement.textContent =
    `Ошибка: ${text}`;
}

```

9. Реализуйте отображение товара

Продолжите ui.js:

```
function createProductElement(
  product
) {
  const card =
    document.createElement("article");

  const title =
    document.createElement("h3");

  title.textContent = product.name;

  const price =
    document.createElement("p");

  price.textContent =
    `Цена: ${product.price} ₺`;

  const category =
    document.createElement("p");

  category.textContent =
    `Категория: ${product.category}`;

  const stock =
    document.createElement("p");

  stock.textContent =
    product.inStock
      ? "В наличии"
      : "Нет в наличии";

  const editButton =
    document.createElement("button");

  editButton.textContent =
    "Изменить цену";

  editButton.dataset.action = "edit";
  editButton.dataset.id = product.id;

  const deleteButton =
    document.createElement("button");
```

```

deleteButton.textContent = "Удалить";
deleteButton.dataset.action = "delete";
deleteButton.dataset.id = product.id;

card.append(
  title,
  price,
  category,
  stock,
  editButton,
  deleteButton
);

return card;
}

```

10. Реализуйте отображение списка

```

export function renderProducts(
  products
) {
  productsContainer.innerHTML = "";

  if (products.length === 0) {
    productsContainer.textContent =
      "Товаров нет";

    return;
  }

  for (const product of products) {
    productsContainer.append(
      createProductElement(product)
    );
  }
}

```

11. Создайте public/js/app.js

Импортируйте функции:

```

import {
  getProducts,
  createProduct,
  updateProduct,
  deleteProduct
}

```

```
} from "./api.js";
```

```
import {  
  renderProducts,  
  showMessage,  
  showError  
} from "./ui.js";
```

12. Реализуйте загрузку товаров

```
async function loadProducts() {  
  try {  
    const products =  
      await getProducts();  
  
    renderProducts(products);  
  } catch (error) {  
    showError(error.message);  
  }  
}
```

13. Загружайте данные при открытии страницы

```
document.addEventListener(  
  "DOMContentLoaded",  
  loadProducts  
);
```

Запустите приложение.

На странице должны появиться товары, которые находятся в backend.

14. Получите форму

```
const form =  
  document.querySelector(  
    "#product-form"  
  );
```

15. Реализуйте создание товара

```
form.addEventListener(  
  "submit",  
  async (event) => {  
    event.preventDefault();  
  
    const product = {  
      name:  
        document  
          .querySelector("#name")
```

```

        .value
        .trim(),

    price:
        Number(
            document
                .querySelector("#price")
                .value
        ),

    category:
        document
            .querySelector("#category")
            .value
            .trim(),

    inStock:
        document
            .querySelector("#inStock")
            .checked
    };

    try {
        await createProduct(product);

        form.reset();

        showMessage(
            "Товар успешно добавлен"
        );

        await loadProducts();
    } catch (error) {
        showError(error.message);
    }
}
);

```

16. Проверьте POST

Через браузерную форму создайте товар:

Название: SSD 2TB

Цена: 65000

Категория: Components

В наличии: да

После отправки формы:

1. frontend должен выполнить POST /api/products;
2. backend должен вернуть 201 Created;
3. форма должна очиститься;
4. список товаров должен обновиться;
5. новый товар должен появиться на странице.

17. Проверьте backend-валидацию

Попробуйте создать товар с ценой:

-100

Frontend должен показать сообщение об ошибке, полученное от API.

Товар не должен добавиться.

18. Реализуйте обработку кнопок списка

В app.js:

```
const productsContainer =  
  document.querySelector(  
    "#products"  
  );
```

```
productsContainer.addEventListener(  
  "click",  
  async (event) => {  
    const button =  
      event.target.closest(  
        "button[data-action]"  
      );  
  
    if (!button) {  
      return;  
    }  
  }
```

```
    const id =  
      Number(button.dataset.id);
```

```
    const action =  
      button.dataset.action;
```

```
    if (action === "delete") {  
      await handleDelete(id);  
    }
```

```
    if (action === "edit") {  
      await handleEdit(id);  
    }
```

```
    }  
  }  
);
```

Такой подход позволяет использовать один обработчик для всех карточек товаров.

19. Реализуйте удаление

```
async function handleDelete(id) {  
  const confirmed =  
    confirm(  
      "Удалить этот товар?"  
    );  
  
  if (!confirmed) {  
    return;  
  }  
  
  try {  
    await deleteProduct(id);  
  
    showMessage(  
      "Товар удалён"  
    );  
  
    await loadProducts();  
  } catch (error) {  
    showError(error.message);  
  }  
}
```

20. Проверьте DELETE

Нажмите кнопку:

Удалить

Ожидаемая последовательность:

Кнопка

↓

data-id

↓

DELETE /api/products/:id

↓

Express API

↓

удаление товара

↓

успешный HTTP-ответ

↓

GET /api/products

↓

обновлённый DOM

Удалённая карточка должна исчезнуть со страницы.

21. Реализуйте простое редактирование цены

Для учебной версии используйте prompt():

```
async function handleEdit(id) {
  const value =
    prompt(
      "Введите новую цену"
    );

  if (value === null) {
    return;
  }

  const price = Number(value);

  if (!Number.isFinite(price)) {
    showError(
      "Цена должна быть числом"
    );

    return;
  }

  try {
    await updateProduct(
      id,
      { price }
    );

    showMessage(
      "Цена изменена"
    );

    await loadProducts();
  } catch (error) {
    showError(error.message);
  }
}
```

22. Проверьте PATCH

Выберите товар и измените его цену.

Frontend должен выполнить запрос вида:

PATCH /api/products/3

Тело:

```
{  
  "price": 42000  
}
```

После успешного ответа карточка должна показывать новую цену.

23. Проверьте ошибку PATCH

Введите:

-500

Backend-валидация должна отклонить запрос.

Frontend должен вывести сообщение об ошибке, а старая цена товара должна сохраниться.

24. Проверьте отсутствие товара

Удалите товар, а затем вручную в консоли браузера выполните запрос к его старому id или вызовите API-функцию с несуществующим id.

Backend должен вернуть:

404 Not Found

Frontend не должен завершаться необработанной ошибкой.

25. Проверьте DevTools

Откройте:

F12 → Network

Для каждой CRUD-операции найдите соответствующий HTTP-запрос:

GET

POST

PATCH

DELETE

Проверьте:

- Request URL;
- Request Method;
- Status Code;
- Request Payload;
- Response.

26. Проверьте полный цикл

Выполните последовательно:

1. Открыть страницу
→ GET /api/products

2. Создать товар
→ POST /api/products
3. Изменить цену
→ PATCH /api/products/:id
4. Удалить товар
→ DELETE /api/products/:id
5. После каждой операции
→ GET /api/products
→ обновление DOM

Требования к результату

Приложение должно:

- открываться через Express;
- использовать каталог public;
- получать товары через REST API;
- не хранить начальный список товаров непосредственно в HTML;
- выполнять GET через fetch();
- отправлять POST из HTML-формы;
- отправлять JSON с Content-Type: application/json;
- выполнять PATCH выбранного товара;
- выполнять DELETE выбранного товара;
- связывать DOM-элемент с ресурсом через id;
- проверять response.ok;
- показывать ошибку пользователю;
- обновлять DOM после изменения серверных данных;
- использовать отдельные api.js, ui.js и app.js.